

Looqbox Challenge - Teste Técnico de Dados

Candidato: Matheus Lucca Ventríci Campos

Data da Entrega: 14/06/2026

Descrição do Desafio:

O desafio foi estruturado para avaliar competências ponta a ponta na manipulação e análise de dados, sendo dividido em duas etapas principais:

1. **SQL Test:** Resolução de três perguntas de negócio utilizando consultas diretas em banco de dados relacional.
2. **Python Cases:** Três desafios específicos de programação, focados em testar o domínio lógico, a capacidade de desenvolver ferramentas dinâmicas e a habilidade de extrair insights através de visualizações analíticas em Python.

Minhas Observações:

Achei o desafio muito bem estruturado. Minha ideia principal foi manter o código o mais simples e limpo possível, sem perder a essência do que estava sendo testado. A parte das consultas em SQL foi mais tranquila de resolver. Já os cases em Python exigiram um pouco mais. Como estou focado em projetar minha carreira na área de dados, fiz questão de ler a documentação e estudei como a biblioteca sqlalchemy funciona por trás dos panos. Foi uma experiência muito bacana.

Resolução do Desafio Técnico de Dados

1. SQL Test

- a. Quais são os 10 produtos mais caros na empresa?

```
SELECT
    PRODUCT_COD,
    PRODUCT_NAME,
    PRODUCT_VAL
FROM
    data_product
ORDER BY
    PRODUCT_VAL DESC
LIMIT 10;
```

- b. Quais seções os departamentos 'BEBIDAS' e 'PADARIA' possuem?

```
SELECT DISTINCT
    DEP_NAME,
    SECTION_NAME
FROM
    data_product
WHERE
    DEP_NAME IN ('BEBIDAS', 'PADARIA')
ORDER BY
    DEP_NAME,
    SECTION_NAME;
```

- c. Qual foi a venda total de produtos (em \$) de cada Área de Negócio no primeiro trimestre de 2019?

```
SELECT
    cadastro.BUSINESS_NAME,
    ROUND(SUM(vendas.SALES_VALUE), 2) AS TOTAL_VENDAS_Q1
FROM
    data_store_sales AS vendas
JOIN
    data_store_cad AS cadastro
    ON vendas.STORE_CODE = cadastro.STORE_CODE
WHERE
    vendas.DATE BETWEEN '2019-01-01' AND '2019-03-31'
GROUP BY
    cadastro.BUSINESS_NAME
ORDER BY
    TOTAL_VENDAS_Q1 DESC;
```

2. Desafios

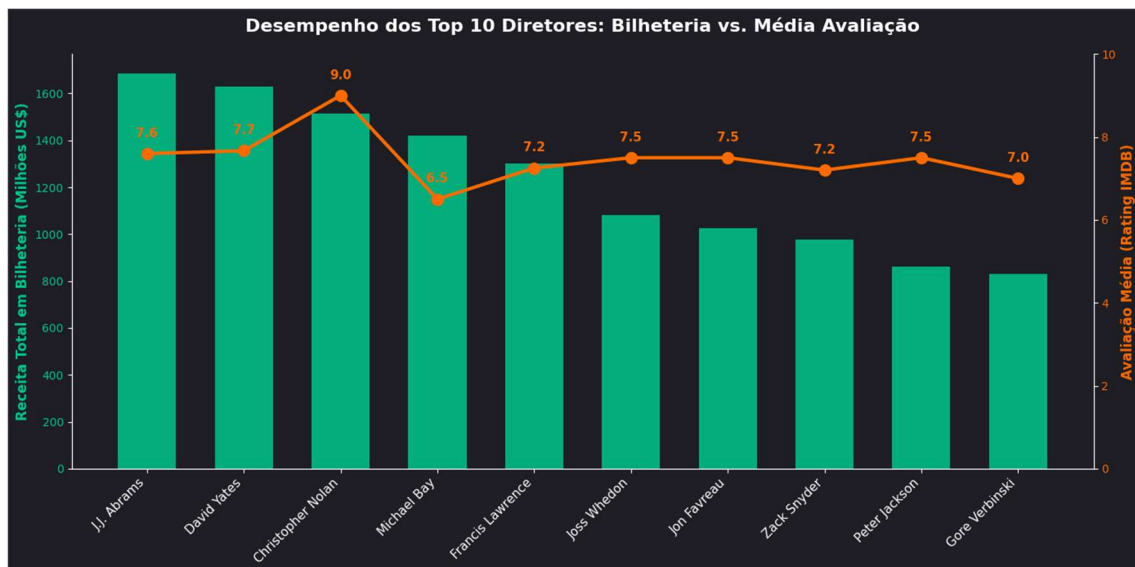
a. Resultado Desafio 1

	STORE_CODE	PRODUCT_CODE	DATE	SALES_VALUE	SALES_QTY
0	10	10	2019-01-01	2386.15	65.0
1	10	10	2019-01-02	4368.49	119.0
2	10	10	2019-01-03	3854.55	105.0
3	10	10	2019-01-04	3671.00	100.0
4	10	10	2019-01-05	3010.22	82.0

b. Resultado Desafio 2

	Loja	Categoria	TM
0	Bahia	Atacado	15.39
1	Bangkok	Posto	13.67
2	Belem	Proximidade	15.37
3	Berlin	Proximidade	15.39
4	Buenos Aires	Atacado	15.39
5	Chicago	Varejo	15.53
6	Dubai	Atacado	15.39
7	Hong Kong	Farma	26.35
8	London	Farma	28.99
9	Madri	Farma	29.03
10	Miami	Posto	13.67
11	New York	Proximidade	15.39
12	Paris	Proximidade	15.39
13	Rio de Janeiro	Farma	29.59
14	Roma	Varejo	15.39
15	Salvador	Atacado	15.39
16	Sao Paulo	Varejo	15.39
17	Sidney	Posto	13.67
18	Tokio	Varejo	15.39
19	Vancouver	Posto	13.67

c. Resultado Desafio 3



3. Utilização de IA.

- a. Utilizei IA apenas para estudar sobre a biblioteca sqlalchemy e escolhas das cores do gráfico, mantive originalmente a estrutura de código que costumo usar.

Após a finalização do teste verifiquei alguns pontos de “segurança” no resultado do desafio 1, utilizei a IA para “suprir” esse erro, entretanto mantive o código “original” que criei, abaixo o código com o ajuste sugerido pela IA:

```
“def retrieve_data_secure(product_code=None,  
store_code=None, date=None):  
    query = "SELECT * FROM data_product_sales WHERE 1=1"  
    parametros = []
```

```
    if product_code is not None:  
        query += " AND PRODUCT_CODE = %s"  
        parametros.append(product_code)
```

```
    if store_code is not None:  
        query += " AND STORE_CODE = %s"  
        parametros.append(store_code)
```

```
    if date is not None and isinstance(date, list) and len(date) == 2:  
        query += " AND DATE BETWEEN %s AND %s"  
        parametros.extend([date[0], date[1]])
```

```
tuple(parametros) resolve a exigência do SQLAlchemy  
df = pd.read_sql(query, engine, params=tuple(parametros))  
  
return df
```

```
meus_dados = retrieve_data_secure(product_code=10,  
date=['2019-01-01', '2019-01-31'])  
display(meus_dados.head())”
```

Em Resumo aprendi que o marcador de posição %s ele é o escudo do código que significa que “vai aparecer um dado aqui”, ou seja a biblioteca substitui o marcador pelo valor real solicitado e aprendi que para solicitar desta maneira temos que utilizar a tupla, para o sqlalchemy por segurança ele acha que estamos fazendo um insert em massa e para a consulta única ele exige que os parâmetros sejam passados em formato de tupla.